ST. DOMINIC COLLEGE OF ASIA
REVOLUTIONIZINGEDUCATION™

# AI-Assisted Multi-Powered Grain Drying System with Android-Based Monitoring & Predictive Control System

A Capstone Project Proposal presented to the Faculty of the

School of Communication, Multimedia & Computer Studies

St. Dominic College of Asia

In Partial Fulfillment of the Requirements for the Degree

Bachelor of Science in Information Technology

Elmar Kenneth Radin, Joshua Santelices,
Prince Jin Stephen Moya, and Mark John Lar

Dr. Alvin Jason Virata

Capstone Advisor

Academic Year 2025–2026

**ACKNOWLEDGEMENT** **3**
**CHAPTER 1**
**INTRODUCTION** **4**
1.1 Background of the Study 4
1.2 Problem Statement 6
1.3 Objectives of the Study 8
General Objective 8
Specific Objectives 8
1.4 Scope and Delimitations 10
1.5 Framework 10
1.6 Significance of the Study 11
1.7 Definition of Terms 12
**CHAPTER 2**
**REVIEW OF RELATED LITERATURE** **15**
Internet of Things (IoT) in Smart Agriculture and Environmental Monitoring 16
Artificial Intelligence and Predictive Analytics in Agricultural Systems 17
Solar-Powered Drying Systems and the Use of Renewable Energy Systems 17
Android-Based Monitoring and Mobile Agricultural Systems 18
**CHAPTER 3**
**RESEARCH METHODOLOGY** **20**
3.1 Project Design 20
3.1.1 System Objective 20
3.1.2 System Framework 21
3.1.3 Feasibility Study 23
3.1.3.1 Technical Feasibility 23
3.1.3.2 Operational Feasibility 25
3.1.3.3 Economic Feasibility 26
3.2 Dataset Development Strategy 28
3.2.1 Data Collection Procedure 28
3.2.2 Dataset Features 28
3.2.3 Dataset Preparation 29
3.3 Machine Learning Model 29
3.4 Development Model 30
3.5 Testing Process. 30
3.6 Installation and Deployment 31
3.7 Maintenance 32

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all individuals who supported us during the preparation and development of this Capstone I project entitled *"AI-Assisted Optimization of IoT Solar-Powered Rice Grain Dryer with Android-Based Monitoring and Predictive Control System."*

First and foremost, we extend our heartfelt appreciation to our adviser, **Dr. Alvin Jason Virata**, for his valuable guidance, technical insights, and constructive feedback throughout the conceptualization and documentation of this study. His expertise greatly helped us refine our system design, research direction, and overall framework.

We also thank the faculty members of the School of Communication, Multimedia, and Computer Studies for equipping us with the knowledge and technical foundation necessary to propose and design this system.

Our gratitude goes to our families for their continuous encouragement, understanding, and moral support as we worked on the planning and documentation phases of this project.

Finally, we acknowledge our team members for their collaboration, shared ideas, and dedication in developing the system proposal and research methodology presented in this Capstone I submission.

To everyone who contributed to this initial stage of our project, we are sincerely thankful.

## CHAPTER 1

# INTRODUCTION

## 1.1 Background of the Study

Agriculture is a cornerstone of food security and economic sustenance in the Philippines, where rice serves as the primary staple crop and accounts for approximately 20 percent of total agricultural output *(Philippine Statistics Authority [PSA], 2023)*. The country produces an estimated 19.8 million metric tons of palay annually, yet post-harvest losses consistently erode both food availability and farmer income. Among post-harvest operations, the drying stage is recognized as the most critical and loss-prone, with improper drying contributing to an estimated 10 to 14 percent of total grain losses nationally *(PhilRice, 2022; Food and Agriculture Organization [FAO], 2021).*

Traditional open-sun drying, which remains the predominant method among smallholder farmers, is inherently dependent on unpredictable environmental variables including solar radiation intensity, ambient humidity, and wind conditions. This dependency produces non-uniform moisture reduction across grain batches, elevates the risk of fungal contamination, particularly from Aspergillus and Fusarium species, and results in inconsistent moisture content at the point of milling. The Philippine Center for Postharvest Development and Mechanization (PHilMECH, 2021) documented that grains dried below the recommended 14 percent wet-basis moisture content threshold experience significant breakage during milling, while those exceeding this threshold during storage develop aflatoxin contamination and premature spoilage.

Solar-powered dryer enclosures have been developed as a practical alternative to uncontrolled open-field drying. By capturing and concentrating solar thermal energy within enclosed chambers, these systems create more controlled drying environments and reduce direct grain exposure to environmental contaminants. Studies by Bala and Debnath (2021) confirm that indirect solar dryers improve drying uniformity and significantly reduce contamination incidence compared to open-sun methods. Despite this advancement, the majority of deployed solar drying systems continue to rely on manual supervision, fixed-threshold relay controls, and purely reactive automation — mechanisms incapable of adapting to the dynamic fluctuations in grain moisture content and ambient conditions that characterize real-world drying environments.

The emergence of the Internet of Things (IoT) has introduced transformative potential in agricultural monitoring. Sensor-equipped embedded systems now enable continuous, real-time acquisition of drying parameters including temperature, relative humidity, airflow velocity, and grain moisture content, which can be transmitted wirelessly to cloud platforms for remote visualization and historical logging. However, existing IoT implementations in post-harvest contexts have largely remained passive monitoring systems — capturing and displaying data without leveraging that data for proactive, predictive decision-making (Liakos et al., 2021; Kumar et al., 2022).

Artificial intelligence, and specifically supervised machine learning, offers a pathway beyond reactive control. Predictive models trained on historical sensor data from actual drying cycles can estimate future moisture behavior, anticipate optimal

drying termination points, and trigger automated system responses — such as fan speed adjustments or heater shutoffs — before conditions become suboptimal rather than after. Liakos et al. (2021), in a comprehensive review published in Sensors, demonstrated the effectiveness of machine learning techniques across agricultural forecasting domains including irrigation optimization and crop yield estimation. Critically, they identified post-harvest drying control as an underexplored area for predictive AI integration — a gap this study directly addresses.

The convergence of renewable energy, IoT sensor technology, and machine learning creates a timely opportunity to design an intelligent grain drying platform suited to the operational realities of Filipino farming communities. This study proposes the development of grAIn — an AI-assisted IoT solar-powered rice grain dryer with Android-based monitoring and predictive control — as a prototype system demonstrating how these technologies can be integrated into a single, deployable solution. The system aims to reduce post-harvest drying losses, improve energy efficiency through solar power, and provide farmers with an intelligent, remotely accessible monitoring and control interface.

## 1.2 Statement of the Problem

Despite advances in agricultural technology, post-harvest rice drying in the Philippines continues to suffer from systemic inefficiencies rooted in manual processes and reactive control systems. These inefficiencies result in measurable grain quality degradation, economic losses for smallholder farmers, and increased food insecurity at a national scale.

Specifically, this study addresses the following problems:

First, traditional sun-drying methods expose rice grains to uncontrolled environmental variability, producing inconsistent moisture reduction and creating conditions for contamination. PHilMECH (2021) estimates that open-sun drying contributes to post-harvest losses of 10 to 14 percent of total grain yield, primarily through uneven drying, over-drying-induced breakage, and moisture-related spoilage during storage.

Second, existing solar-powered dryer implementations, while offering more controlled environments than open-field drying, rely predominantly on static threshold-based automation. These systems respond reactively to sensor readings that exceed predefined limits rather than anticipating optimal intervention points based on learned drying behavior. Kumar et al. (2022) established that dynamic airflow and temperature regulation — driven by real-time drying trend analysis — is essential for achieving consistent moisture uniformity across grain batches.

Third, current IoT-enabled agricultural monitoring systems primarily serve as data visualization platforms rather than predictive decision-support tools. While sensor data is captured and displayed in real time, the intelligence needed to translate that data into proactive control actions — such as predicting the remaining drying time or automatically terminating the drying cycle at optimal moisture content — remains largely absent in deployed systems (Liakos et al., 2021).

Fourth, the absence of an integrated mobile decision-support interface limits farmer accessibility and real-time responsiveness. Remote monitoring and control capabilities are essential for smallholder farmers who cannot maintain continuous physical presence at the drying facility, particularly during extended or overnight drying cycles.

Fifth, no standardized public dataset exists for training predictive moisture estimation models specific to solar-powered rice grain drying in Philippine conditions. This data gap has historically constrained the development of context-specific machine learning models capable of accurate termination point prediction.

Taken together, these problems represent a clear need for an integrated system that combines solar-powered drying, multi-sensor IoT monitoring, AI-driven predictive control, and mobile-based remote supervision into a unified, deployable prototype.

## 1.3 Objectives of the Study

### General Objective

To design, develop, and evaluate an AI-assisted IoT solar-powered rice grain dryer with Android-based monitoring and predictive control that optimizes drying performance, reduces post-harvest losses, and improves energy efficiency through machine learning-driven automated control.

**Specific Objectives**

- To design and implement a multi-sensor IoT data acquisition system using temperature (DHT22), humidity, grain moisture, load-cell weight, and solar voltage sensors capable of collecting continuous drying parameters at configurable sampling intervals of 3 to 30 seconds.
- To develop and train a supervised Random Forest machine learning model using a synthetically generated drying dataset based on Page's thin-layer drying equation and Henderson-Thompson equilibrium moisture parameters, producing a predictive model with a target moisture estimation $R^2$ of at least 0.90 on held-out test data.
- To implement an AI-driven predictive control algorithm that dynamically regulates fan speed, heater activation, and stirring mechanism based on real-time sensor inputs and predicted moisture trajectories to achieve uniform drying to the 14 percent wet-basis safe storage threshold.
- To integrate a solar photovoltaic subsystem as the primary energy source for the drying prototype, with solar voltage telemetry included as a feature variable in the predictive model to support energy-aware drying optimization.
- To develop a React Native Android mobile application that provides real-time sensor visualization, remote dryer control, drying session tracking, AI prediction display, and push notification alerts for critical drying events.

- To develop a Next.js web admin dashboard for system management, device monitoring, historical analytics, and session oversight, connected to the IoT backend via Server-Sent Events for real-time updates.
- To evaluate the implemented system's predictive accuracy, control responsiveness, and usability through functional testing, integration testing, and a structured User Acceptance Testing instrument administered to target farmer and technologist evaluators.

## 1.4 Scope and Delimitations

This study covers the design, development, and prototype-scale evaluation of an AI-assisted IoT solar-powered rice grain dryer. The scope encompasses the following components and activities:

- Hardware design and assembly of the drying prototype incorporating ESP32 and Arduino UNO microcontrollers, DHT22 temperature-humidity sensors, capacitive grain moisture sensors, HX711 load-cell modules, relay-controlled fans and heater, stepper motor stirring mechanism, and a solar photovoltaic panel with INA219 energy monitoring.
- Development of a Next.js full-stack backend API serving as the central data gateway, command dispatcher, Firebase Realtime Database synchronization layer, and machine learning prediction proxy.

- Development of a Python Flask ML microservice hosting a trained Random Forest model for real-time moisture prediction and drying termination estimation.
- Development of a React Native Expo Android mobile application providing real-time monitoring, remote dryer control, AI prediction visualization, drying session management, and push notification delivery.
- Development of a Next.js web admin dashboard for device management, user administration, analytics, and real-time SSE-based monitoring.
- Generation of a synthetic training dataset of 22,753 samples modeled on Page's thin-layer drying equation, to be supplemented by real drying cycle data collected during prototype testing.
- Functional, integration, and User Acceptance Testing within controlled laboratory conditions at St. Dominic College of Asia.

The following are explicitly excluded from the scope of this study:

- Large-scale commercial or industrial deployment. The prototype is designed for evaluation-scale grain batches of approximately 2 to 5 kilograms.
- Native iOS application development. The mobile platform target is Android exclusively, though the Expo framework used is cross-platform capable.

- Long-term field trials under varying Philippine regional climate conditions. Testing is conducted within controlled laboratory settings.
- Sensor calibration certification to commercial or regulatory standards. Sensor accuracy is evaluated relative to reference instruments within the scope of prototype validation.
- Retraining of the machine learning model on large-scale industrial drying datasets. The current model is validated on synthetic and limited experimental data and is intended as a proof-of-concept implementation.
- Agricultural policy integration or government regulatory compliance assessment.

## 1.5 Conceptual Framework

The study is guided by an Input-Process-Output (IPO) framework that defines the flow of data, control signals, and system outputs across the integrated platform. Figure 1.1 presents the conceptual framework of the grAIn system.

INPUT: The system receives multi-sensor data from the drying environment — including temperature (°C), relative humidity (%), grain moisture content (%), fan speed (%), solar voltage (V), grain weight (kg), and energy consumption (kWh) — transmitted by the ESP32 microcontroller over WiFi at configurable intervals. User commands entered through the Android mobile application or the web admin dashboard also constitute inputs to the system.

PROCESS: Raw sensor data is validated, persisted to MongoDB Atlas, and simultaneously synchronized to Firebase Realtime Database for low-latency mobile delivery. The AI prediction microservice receives sensor readings and applies the trained Random Forest model to generate a 30-minute moisture forecast, estimated time-to-target, efficiency score, and control recommendation. The predictive control algorithm interprets these outputs to dynamically regulate the fan speed, heater state, and stirring mechanism. Automated alerts are generated when sensor readings exceed safety thresholds, and drying session metrics — including energy consumed, average temperature, and drying efficiency — are continuously updated throughout each session.

OUTPUT: The system produces dried grain batches terminating at the 14 percent moisture content safe storage threshold, a real-time monitoring dashboard accessible via Android and web interfaces, historical drying session records with efficiency analytics, AI-generated drying recommendations and projected moisture curves, and push notification alerts for critical events including drying completion, temperature warnings, and device offline conditions.

## 1.6 Significance of the Study

The findings and prototype system produced by this study are significant to the following stakeholder groups:

Smallholder Rice Farmers. The primary beneficiaries of the proposed system are smallholder rice farmers who currently rely on manual, weather-dependent drying methods. The grAIn system provides an affordable, remotely accessible platform for monitoring and controlling grain drying operations, reducing grain losses attributable to over-drying and under-drying, and improving overall grain quality at the point of milling and storage.

Agricultural Extension Workers and Technologists. The system demonstrates a replicable model for integrating IoT, AI, and mobile technology into post-harvest agricultural equipment. Agricultural extension workers can reference this study as evidence of technical feasibility for similar deployments in provincial and rural contexts.

Academic Researchers. This study contributes to the limited body of published research on AI-driven predictive control in post-harvest rice drying, specifically for Philippine agricultural conditions. The synthetic dataset generation methodology using Page's equation, the Random Forest model architecture, and the system integration architecture are documented as reproducible contributions to the field.

Software and Hardware Developers. The open-architecture system design — spanning ESP32 firmware, Next.js API, Python ML microservice, React Native mobile app, and Firebase/MongoDB cloud integration — provides a documented technical reference for developers working on similar IoT agricultural automation projects.

Environmental and Renewable Energy Sector. By demonstrating the viability of solar photovoltaic energy as a primary power source for intelligent grain drying, this study supports the case for renewable energy adoption in post-harvest agricultural

infrastructure, contributing to national sustainability goals outlined in the Philippine Energy Plan.

St. Dominic College of Asia. This capstone project demonstrates the applied engineering and AI capabilities of the BSIT program, serving as a reference system for future capstone and thesis projects in IoT, machine learning, and smart agriculture.

## 1.7 Definition of Terms

The following terms are defined as they are used in this study:

Airflow Regulation Mechanism. A control component that adjusts fan speed and air circulation direction within the drying chamber to maintain uniform heat distribution and optimal drying rate. In this system, airflow regulation is dynamically adjusted based on predictive model outputs.

Android-Based Monitoring System. The React Native mobile application developed in this study, targeting the Android platform, that provides real-time sensor visualization, remote dryer control, AI prediction display, drying session management, and push notification delivery.

Data Acquisition System (DAQ). The ESP32 and Arduino UNO microcontroller assembly responsible for reading sensor data, executing actuator commands, and transmitting readings to the backend API via WiFi HTTP requests.

Drying Session. A discrete drying cycle initiated by the user, tracked from grain loading to the point at which grain moisture content reaches the target threshold. Sessions record start moisture, end moisture, energy consumed, average temperature and humidity, fan speed, and drying efficiency.

Energy Efficiency. In the context of this study, the ratio of effective moisture reduction achieved to total energy consumed during a drying session, used as a key performance metric for system evaluation.

Equilibrium Moisture Content (EMC). The moisture content at which grain neither gains nor loses moisture to the surrounding air, modeled using the Henderson-Thompson equation for rough rice: EMC = [ln(1 - RH) / (-A × (T + C))]^(1/B), where A, B, and C are grain-specific constants.

grAIn. The system name assigned to the prototype developed in this study, representing the AI-assisted IoT solar-powered rice grain dryer platform. The name reflects the system's agricultural purpose and its core AI-driven optimization capability.

Internet of Things (IoT). A distributed network of physical devices embedded with sensors, communication modules, and software that collect and exchange data over the internet. In this study, IoT refers specifically to the ESP32-based sensor and actuator network connected to the cloud backend.

Machine Learning (ML). A subset of artificial intelligence in which statistical models are trained on historical data to learn patterns and make predictions or decisions without being explicitly programmed for each specific case. This study employs supervised machine learning for grain moisture prediction.

Moisture Content (MC). The percentage of water present in rice grain, expressed on a wet basis as: MC (%) = (mass of water / total mass of grain) × 100. The safe storage threshold for Philippine rice varieties is 14 percent wet basis (PhilRice, 2022).

Over-Drying. Reduction of grain moisture content below the optimal 14 percent threshold, resulting in grain brittleness, increased breakage during milling, and reduced head rice recovery.

Page's Drying Equation. A thin-layer drying model describing moisture ratio (MR) as a function of time: MR = exp(-k × t^n), where MR = (M - Me) / ($M_0$ - Me), M is current moisture, Me is equilibrium moisture, $M_0$ is initial moisture, k is the drying constant, n is Page's exponent, and t is elapsed drying time. This equation forms the physical basis for synthetic training data generation in this study.

Predictive Control System. An advanced automation strategy in which future system behavior is estimated from current and historical data, enabling preemptive control actions rather than reactive responses to threshold violations. In this system, predicted moisture trajectories trigger automated fan, heater, and termination decisions.

Random Forest Regressor. An ensemble machine learning algorithm that constructs multiple decision trees during training and outputs the mean prediction of individual trees for regression tasks. The model used in this study comprises 200 estimators with a maximum depth of 20.

Solar Photovoltaic (PV) Subsystem. The solar panel and INA219 current-voltage monitoring module integrated into the drying prototype as its primary energy source. Solar voltage is monitored as a real-time input to the AI prediction model.

Threshold-Based Control. A reactive automation method in which system actions are triggered when a measured parameter crosses a predefined fixed limit. Contrasted with predictive control in this study.

Under-Drying. Insufficient moisture reduction leaving grain above the 14 percent safe storage threshold, increasing susceptibility to mold growth, aflatoxin contamination, and storage spoilage.

User Acceptance Testing (UAT). A structured evaluation process in which target end-users assess whether the system meets functional requirements and usability expectations. In this study, UAT is administered using a validated Likert-scale instrument to farmer and technologist evaluators.

*— End of Chapter 1 —*

# CHAPTER 2

# REVIEW OF RELATED LITERATURE

This chapter presents a review of relevant literature and studies related to the technologies utilized in the proposed system. The discussion focuses on Internet of Things (IoT) applications in agriculture, artificial intelligence for predictive control, solar-powered drying technologies, and Android-based monitoring systems. These studies provide the theoretical and technological foundation for the development of the AI-assisted IoT solar-powered rice grain dryer with Android-based predictive monitoring and control.

**Sectors:** Agriculture (Rice Farming and Post-Harvest Processing)

**Problem Addressed:** The main challenges are significant post-harvest losses resulting from inefficient drying processes, a reliance on unpredictable weather conditions, the lack of automated predictive capabilities, and the absence of integrated monitoring systems.

**General scope:** The proposed system utilizes multiple sensors to monitor environmental parameters such as temperature, humidity, airflow, and grain moisture content. Artificial intelligence is applied for predictive modeling of drying behavior, while solar-powered technology serves as the primary energy source for the drying system. Automated airflow regulation is implemented to maintain

optimal drying conditions, and an Android-based real-time dashboard is developed for monitoring system performance and receiving alerts.

## Internet of Things (IoT) in Smart Agriculture and Environmental Monitoring

### A. Concepts and Definitions

The Internet of Things (IoT) refers to interconnected physical devices embedded with sensors and communication capabilities that enable automated data collection and exchange. In agriculture, IoT systems typically consist of environmental sensors, microcontrollers, gateways, cloud platforms, and dashboards. According to Ray (2021), IoT enables real-time monitoring of environmental variables such as temperature and humidity, which improves operational efficiency and decision-making accuracy. Wireless communication technologies including Wi-Fi, GSM, and LoRaWAN ensure stable data transmission in rural agricultural settings (Sadowski & Spachos, 2020).

### B. IoT for Post-Harvesting Monitoring

IoT has been increasingly applied in post-harvest systems to monitor grain moisture, ambient humidity, and temperature conditions during drying and storage. Elijah et al. (2021) emphasize that real-time environmental monitoring reduces post-harvest losses by providing early detection of unfavorable conditions. Zhang et al. (2022) further highlight that agricultural IoT platforms improve traceability and transparency in farm operations. However, most systems remain limited to monitoring functions without intelligent automation.

### C. Real-Time Data Acquisition and Reliability

Reliable data acquisition is critical in smart agricultural systems. Continuous sensing ensures accurate monitoring of environmental conditions inside drying chambers. According to Khan et al. (2023), data reliability and low latency are essential for automated decision systems. Rural deployment also requires stable communication protocols and energy-efficient sensor designs to maintain uninterrupted operation.

### D. Previous IoT-Based Agricultural Drying Systems

Existing IoT-based drying prototypes focus mainly on temperature and humidity monitoring. Liakos et al. (2021) explain that while IoT enhances monitoring capabilities, many implementations lack predictive analytics and automated control mechanisms. This limitation results in systems that assist observation but still rely heavily on manual decision-making.

## Artificial Intelligence and Predictive Analytics in Agricultural Systems

### A. Artificial Intelligence in Agriculture Automation

Artificial Intelligence (AI) uses machine learning algorithms to analyze environmental data and generate predictive insights. Liakos et al. (2021) report that AI applications in agriculture include irrigation scheduling, yield prediction, and environmental forecasting. These systems improve efficiency by minimizing human intervention and optimizing resource utilization.

### B. Predictive Modeling for Moisture Content

Predictive modeling techniques can estimate moisture reduction trends during grain drying. Smith et al. (2022) found that machine learning-based drying models improve energy efficiency by predicting optimal drying termination points. This prevents unnecessary heating and reduces operational costs.

### C. Risk of Over-Drying and Under-Drying

Improper drying leads to either over-drying, which causes grain cracking and reduced milling recovery, or under-drying, which increases the risk of fungal growth and spoilage. Bala and Debnath (2021) emphasize the importance of controlled drying environments to maintain grain quality. AI-based monitoring can help prevent these risks by maintaining optimal thresholds.

### D. AI-Based Decision Support Systems

AI-based decision systems translate sensor data into automated control actions such as activating ventilation fans or adjusting airflow. Khan et al. (2023) explain that predictive decision support improves system responsiveness and reduces human error. However, integration of AI into small-scale solar-powered dryers remains limited.

## Solar-Powered Drying Systems and the Use of Renewable Energy Systems

### A. Solar Dryer Technologies

Solar drying systems such as cabinet dryers and indirect solar dryers provide controlled environments that protect grains from contamination. Bala and Debnath (2021) highlight that solar dryers improve drying efficiency compared to traditional sun drying.

### B. Hybrid Drying Systems

Hybrid systems integrate solar collectors with mechanical airflow systems to enhance moisture removal. Kumar et al. (2022) explain that optimized airflow significantly improves drying uniformity and reduces processing time.

### C. Energy Efficiency and Sustainability

Renewable energy integration reduces operational costs and carbon emissions. The Food and Agriculture Organization (2021) emphasizes renewable-based agricultural technologies as essential for sustainable development and climate resilience.

### D. Limitation of Existing Solar Dryers

Despite technological advancements, many solar dryers still rely on manual supervision. They lack predictive automation and mobile monitoring integration, limiting their efficiency and usability for smallholder farmers.

## Android-Based Monitoring and Mobile Agricultural Systems

### A. Mobile-Based Farm Monitoring Systems

Mobile applications allow remote monitoring of agricultural systems through real-time dashboards. Javaid et al. (2023) state that Android-based platforms are widely adopted due to affordability and accessibility.

### B. UI and Human-Computer Interaction

Effective agricultural applications utilize simple layouts, graphical data representation, and alert notifications to enhance usability. Khan et al. (2023) note that user-friendly dashboards increase farmer adoption rates.

### C. Mobile-Based Agriculture Case Studies

Several agricultural mobile applications focus on irrigation management and crop monitoring. However, limited studies address mobile-integrated predictive drying control systems.

### D. Identified Gap in Mobile Drying Systems

Most mobile agricultural systems provide monitoring only. There remains a lack of AI-driven automated drying control integrated into Android-based applications.

# CHAPTER 3

# RESEARCH METHODOLOGY

This chapter presents the system development methodology used in designing and implementing the AI-Assisted Optimization of IoT Solar-Powered Rice Grain Dryer with Android-Based Monitoring and Predictive Control System. The methodology outlines the system objectives, framework, feasibility analysis, development model, dataset strategy, testing procedures, deployment process, and maintenance plan. All processes are aligned with the objectives presented in Chapter I.

## 3.1 Project Design

### 3.1.1 System Objective

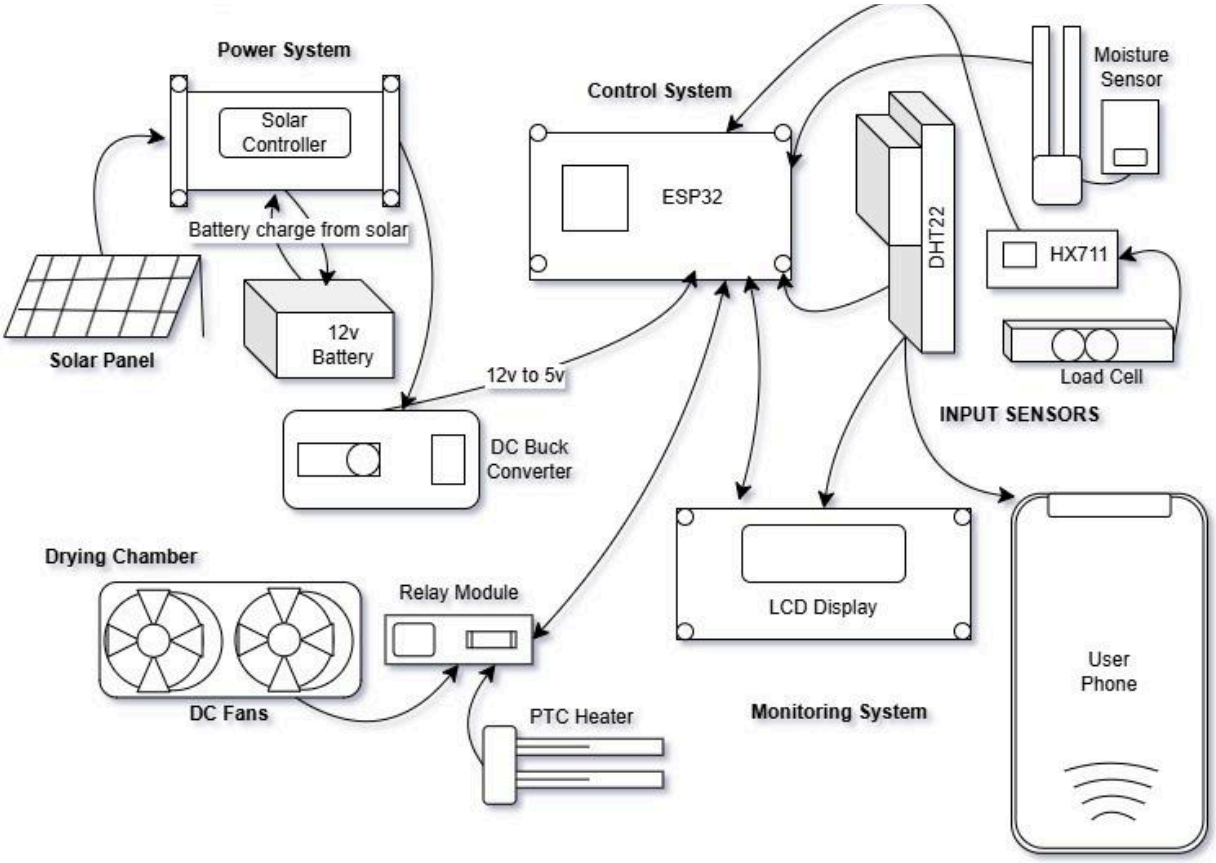
The primary objective of this project is to design and develop an AI-assisted IoT solar-powered rice grain dryer capable of real-time environmental monitoring, predictive moisture estimation, and automated airflow regulation.

Specifically, the system aims to:

- Detect and log real-time environmental parameters including temperature, humidity, airflow, solar voltage, and grain moisture content.
- Generate an experimental dataset from multiple drying cycles.
- Train and validate a supervised machine learning model for moisture prediction and termination point estimation.

- Implement a predictive control algorithm to dynamically regulate airflow.
- Provide an Android-based monitoring application for real-time visualization and alerts.
- Compare predictive drying control against conventional threshold-based control.

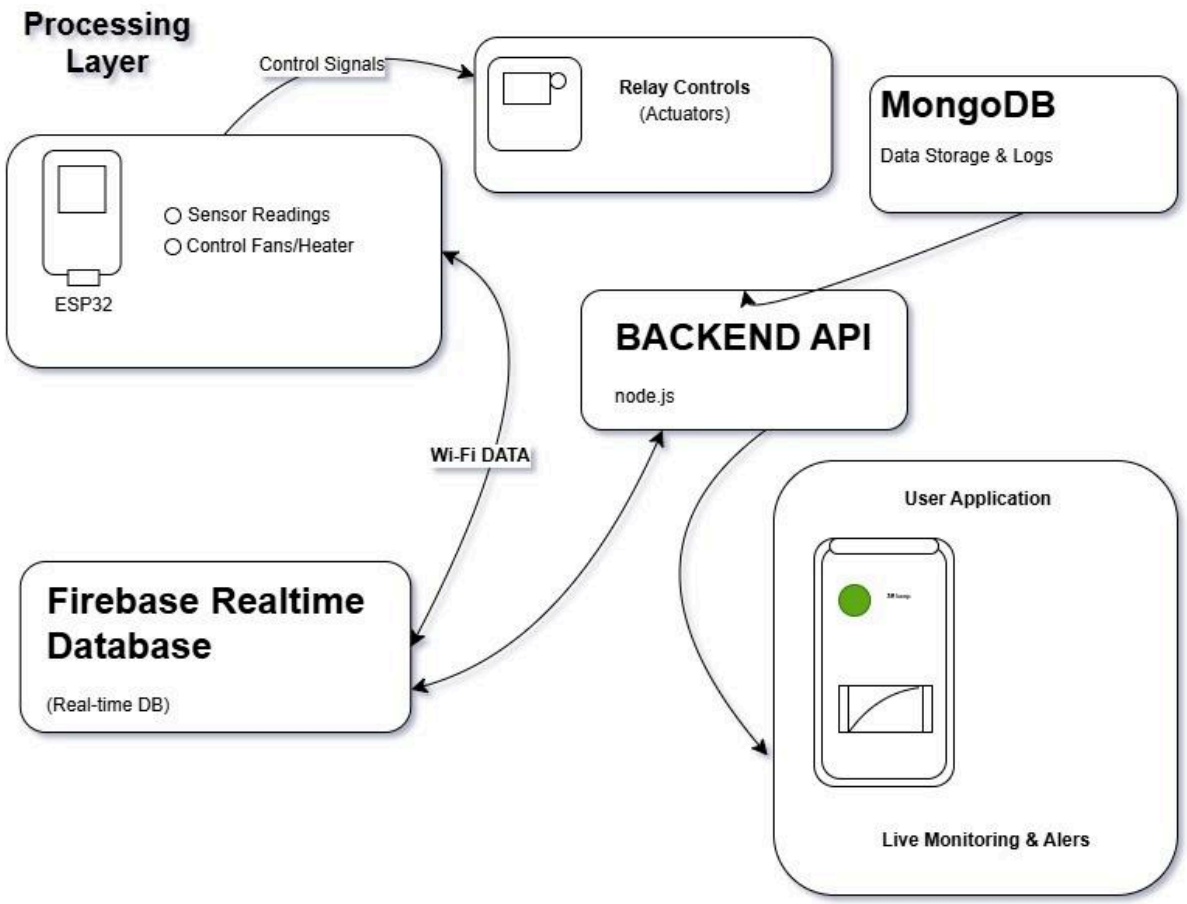
### 3.1.2 System Framework




*Hardware Layer*

The Input phase consists of all data and resources required for the operation of the smart rice grain dryer. These include real-time temperature readings captured by the thermal sensor, relative humidity measurements, airflow rate values, grain moisture content readings, and solar panel voltage output. Additionally, predefined moisture thresholds (e.g., 14% safe storage level) and user-generated configuration inputs from the Android application are incorporated. The hardware inputs are acquired through sensors connected to the ESP32 microcontroller, which gathers

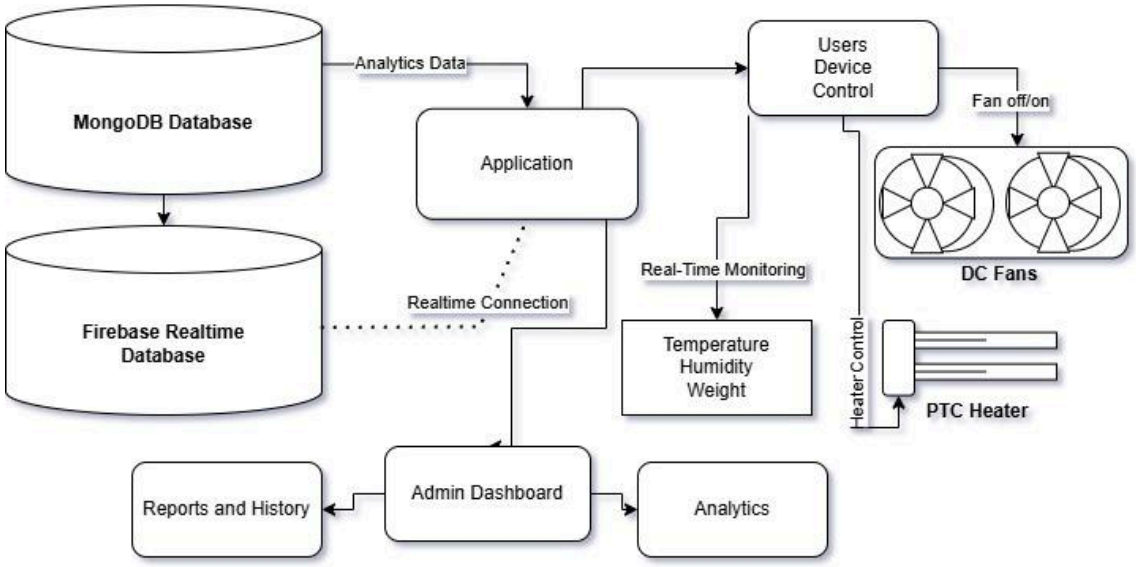
environmental data from within the drying chamber. Network connectivity serves as an operational input, enabling the transmission of processed data to the Firebase cloud database. Together, these inputs provide the foundation for the system's predictive and automated control functions.




*Processing Layer*

The Process phase involves the transformation of raw sensor data into actionable drying control information. Initially, the ESP32 microcontroller collects readings from the temperature, humidity, airflow, and moisture sensors. These

measurements are processed and transmitted via Wi-Fi to the Firebase Cloud Database, where dataset logging and storage occur. The system then applies a machine learning model to infer predictive control decisions. Based on the current drying conditions, the model estimates the remaining drying time and expected final moisture content. The ESP32 microcontroller uses this information to adjust fan speeds, airflow rates, or other operational parameters, ensuring the drying process remains optimal. Decisions may include continuing drying, reducing airflow, or terminating the process when the target moisture content is achieved.




*Application Layer*

The Output phase represents the observable results of the system's data processing and control operations. These outputs include real-time drying parameters displayed in the Android application, predicted drying completion status, automated airflow adjustments, and alert notifications when drying is complete. Comparative performance results between AI-driven control and threshold-based control are also

provided. Collectively, these outputs demonstrate the functional implementation of the IoT-based predictive rice grain dryer, offering accurate, real-time, and remotely accessible monitoring and control of the drying process.

### 3.1.3 Feasibility Study

#### 3.1.3.1 Technical Feasibility

The system is technically feasible based on the availability of required hardware and software components.

| Component | Specification | Purpose | Quantity |
|---|---|---|---|
| ESP32 Microcontroller | **ESP32**, Wi-Fi + Bluetooth, 240 MHz dual-core, 3.3 V logic | Main controller that collects sensor data, processes readings, controls fans/actuators, and sends data to the cloud | 1 |
| Temperature & Humidity Sensor | **DHT22** or equivalent digital sensor | Measures internal dryer temperature and humidity for monitoring and automatic control of drying conditions | 5-6 |
| Grain Moisture Sensor | Capacitive/agricultural moisture | Detects rice grain moisture level to | 1 |

| | | | |
|---|---|---|---|
| | probe, 0–100% moisture range | determine drying progress and completion | |
| Load Cell + Amplifier | 5–20 kg load cell with **HX711** | Measures grain weight reduction to estimate water loss during drying | 1 set |
| Airflow Control Fan | 12 V DC brushless fan (high airflow type) | Circulates hot air inside the chamber for uniform drying | 2 |
| Exhaust Fan | 12 V DC fan | Removes moist air from the chamber | 1 |
| Solar Panel | 100–150 W mono/polycrystalline panel | Converts sunlight into electrical energy for system power | 1 |
| Battery Storage | 12 V deep-cycle lead-acid or $LiFePO_4$ (20–40 Ah) | Stores solar energy to power system during low sunlight or nighttime | 1 |
| Charge Controller | 12 V PWM/MPPT solar charge controller | Regulates charging and protects battery from overcharge/discharge | 1 |
| Cloud Database | **Firebase Realtime Database, MongoDB** | Stores real-time sensor data for remote monitoring and history logs | 1 (cloud service) |

| | | | |
|---|---|---|---|
| Mobile App Development | **Visual Studio Code** | Used to build Android app for monitoring and control | 1 (software tool) |
| Machine Learning Platform | **Python** with ML libraries | Trains predictive drying model and performs moisture prediction/inference | 1 (software environment) |
| Dryer structure/Chamber | Wood/metal frame, acrylic top cover, mesh tray, insulated walls | Physical enclosure that traps heat and holds grains for drying | 1 |

All components are commercially available and affordable. Machine learning model training will be conducted using Python, and the final trained model will be deployed for inference within the system.

### 3.1.3.2 Operational Feasibility

The system is operationally feasible for small- to medium-scale rice farmers.

- The Android application will use color-coded indicators for drying status.
- The drying process is largely automated.
- Manual override will be available if needed.
-

- No advanced technical expertise is required for daily operation.

Initial testing will be conducted in a controlled prototype environment simulating real drying conditions.

### 3.1.3.3 Economic Feasibility

| Component | Quantity | Estimated Cost (₱) | Subtotal (₱) |
|---|---|---|---|
| **Microcontroller & Communication** | | | |
| Arduino Uno R3 | 1 | 380 | 380 |
| ESP8266-01 (or NodeMCU) | 1 | 400 | 400 |
| **Sensors** | | | |
| DHT22 Temperature & Humidity Sensor | 1 | 250 | 250 |
| Soil Moisture Sensor (capacitive type) | 1 | 500 | 500 |
| HX711 + Load Cell Set (10kg) | 1 set | 500 | 500 |
| **Actuators** | | | |
| DC Air Circulation Fan (Intake) - 12V | 1 | 250 | 250 |
| DC Air Circulation Fan (Exhaust) - 12V | 1 | 250 | 250 |
| PTC Heater (12V, 100W) | 1 | 300 | 300 |
| **Relays** | | | |
| 4-Channel Relay Module (5V) | 1 | 250 | 250 |
| Single Relay Module (5V) | 1 | 100 | 100 |
| **Stepper Motor** | | | |

| | | | |
|---|---|---|---|
| 28BYJ-48 Stepper Motor + ULN2003 Driver | 1 set | 300 | 300 |
| **Display** | | | |
| LCD 16x2 with I2C Module | 1 | 250 | 250 |
| **Buzzer** | | | |
| 5V Active Buzzer | 1 | 50 | 50 |
| **Power System** | | | |
| Solar Panel (100-150W) | 1 | 3,000 | 3,000 |
| 12V Deep Cycle Battery | 1 | 2,000 | 2,000 |
| Solar Charge Controller | 1 | 600 | 600 |
| LM2596 DC-DC Buck Converter (5V for Arduino/ESP) | 1 | 150 | 150 |
| **Mechanical Structure** | | | |
| Dryer Structure (wood/metal frame, acrylic cover, trays, insulation) | 1 set | 5,000 | 5,000 |
| **Miscellaneous** | | | |
| Wires, connectors, screws, PCB, casing, resistors (4.7kΩ, 10kΩ) | 1 lot | 1,000 | 1,000 |
| **TOTAL** | | | **₱15,930** |

***Table 1 presents the estimated prototype cost.***

Compared to commercial mechanical dryers costing significantly more, the proposed system offers a cost-efficient alternative.

Payback potential includes:

- Reduced drying time
- Reduced grain losses
- Reduced labor monitoring
- Improved grain quality

### 3.2 Dataset Development Strategy

Since no public dataset exists for solar-powered rice drying predictive optimization, the dataset will be generated experimentally.

#### 3.2.1 Data Collection Procedure

- Rice grains will be loaded with measured initial moisture content.
- Sensors will log parameters every 1 minute.
- Each drying cycle is expected to last 5–8 hours.
- Minimum of 20–30 full drying cycles will be conducted.

Estimated dataset size:

6,000–15,000 records.

#### 3.2.2 Dataset Features

Independent Variables:

- Temperature
- Relative Humidity
- Airflow Speed

- Solar Voltage
- Time Elapsed

Dependent Variable:

- Grain Moisture Content

Optional Classification Label:

- Drying Complete (Yes/No)

### 3.2.3 Dataset Preparation

- Missing value removal
- Outlier detection
- Feature normalization
- 80% Training / 20% Testing split
- Cross-validation

## 3.3 Machine Learning Model

Two regression models will be evaluated:

1. Multiple Linear Regression (baseline)
2. Random Forest Regressor (non-linear modeling)

Evaluation Metrics:

- Mean Absolute Error (MAE)
- Root Mean Square Error (RMSE)
- $R^2$ Score

Model selection will be based on lowest error and highest stability.

### 3.4 Development Model

The project follows a Modified Waterfall Model:

1. Requirements Analysis
2. System Architecture Design
3. Hardware Assembly
4. Dataset Collection
5. Model Training & Validation
6. Android Integration
7. Testing & Optimization
8. Deployment

### 3.5 Testing Process.

| Test Type | Description | Expected Result |
|---|---|---|
| Unit Testing | Sensor calibration accuracy | ± acceptable tolerance |
| Integration Testing | ESP32 to Firebase data sync | Latency < 2 sec |
| Model Validation | Prediction error evaluation | MAE minimized |

| | | |
|---|---|---|
| Comparative Testing | AI vs threshold control | AI shows improvement |
| User Acceptance Testing | Farmer usability survey | Mean ≥ 4.0 rating |

Performance indicators:

- Drying time reduction
- Over-drying reduction
- Moisture variance consistency
- Energy usage efficiency

### 3.6 Installation and Deployment

Deployment will follow:

1. Hardware assembly and sensor calibration
2. Firmware upload via Arduino IDE
3. Firebase database schema configuration
4. Android APK deployment
5. End-to-end integration test
6. Controlled drying trial execution

### 3.7 Maintenance

Maintenance includes:

Hardware:

- Monthly sensor cleaning
- Moisture sensor recalibration
- Solar panel inspection

Software:

- Firebase monitoring
- Model retraining (if dataset expands)
- Android app updates

# CHAPTER 4

## RESULTS AND DISCUSSION

*DRAFT NOTE (delete before submission): text highlighted in yellow marks data that must be replaced or confirmed. Values without highlight come from the Capstone 1 documentation, the grAIn-ml-service training records, and the hardware test write-up.*

*HARDWARE DATA NOTE (delete before submission): Sections 4.1.4, 4.2, and 4.5 are transcribed from the hardware engineer's test write-up. Check every value against the raw logs, photos, or notebook before submission, and submit only measurements that were actually taken. Items marked CONFIRM are inconsistencies inside that write-up.*

This chapter presents the results of the development and evaluation of the grAIn system, an AI-assisted multi-powered grain drying system with Android-based monitoring and predictive control. The results are organized into seven parts: (4.1) implementation of the panel-required revisions, (4.2) hardware implementation, (4.3) model development results, (4.4) model validation using completed drying sessions, (4.5) system performance and integration testing, (4.6) ISO/IEC 25010 evaluation by experts and end-users, and (4.7) discussion of the results against the study objectives.

### 4.1 Implementation of Panel-Required Revisions

Following the Capstone 1 panel review, five functional revisions were implemented, together with a change of the development approach from Waterfall to Agile Iterative. Table 4.1 summarizes the state of the system before and after each revision.

**Table 4.1. Summary of implemented revisions**

| Revision | Before | After | Benefit |
|---|---|---|---|
| 1. Multi-model AI prediction | Random Forest only | Gradient Boosting and the Modified Chung–Pfost EMC model added alongside Random Forest | Allows comparison of prediction approaches; combines data-driven and physics-based estimates; stronger basis for estimating drying time and moisture behavior |
| 2. Predictive control integration | Mostly reactive: heater and fans responded to current temperature | Predicted drying rate and remaining drying time integrated into the control process | Moves from reactive to predictive control; heating and fan operation follow expected drying behavior |

| Revision | Before | After | Benefit |
|---|---|---|---|
| 3. Moisture data integration | Moisture data not properly integrated | Software pipeline from moisture input to backend, database, and mobile application; no physical moisture sensor in the current hardware scope | Moisture can be monitored, stored, and used in prediction and control; ready for full moisture-sensor integration |
| 4. Automatic fan adjustment | Simple ON/OFF relay control | Fan operation determined from current sensor readings and predicted drying condition; relay kept for ON/OFF power switching | Automatic response to changing conditions; suitable airflow per drying stage; less manual adjustment |
| 5. Model validation | Individual test runs on limited data | Predicted drying time compared with actual drying time of completed sessions | Measurable accuracy; evidence of predictive effectiveness |

### *4.1.1 Multi-Model AI Prediction*

The ML microservice (grAIn-ml-service, Python Flask) now hosts three prediction approaches. The Random Forest Regressor (200 estimators, maximum depth 20) predicts moisture 30 minutes ahead from nine input features. A Gradient Boosting model [confirm: Gradient Boosting or XGBoost; add hyperparameters] serves as a second data-driven estimator. The Modified Chung–Pfost equation serves as the physics-based equilibrium moisture content (EMC) model: $Me = E - F \cdot \ln[-(T + C) \cdot \ln(RH)]$, where Me is the equilibrium moisture content, T is temperature, RH is relative humidity, and E, F, and C are grain-specific constants [insert constants used in the code and confirm the equation form matches the implementation]. The Henderson–Thompson equation remains in use for generating the synthetic training data. [Describe how the three outputs are combined: weighted ensemble, best-model selection, or shown side by side.]

[INSERT SCREENSHOT: AI Prediction screen showing projected moisture curve and model outputs]

*Figure 4.1. AI prediction screen of the grAIn mobile application*

### *4.1.2 Predictive Control Integration*

The predictive control loop runs inside the sensor-ingest endpoint of the grAIn-admin backend on every reading posted by the ESP8266, because the Render.com free tier does not provide persistent background workers. In AUTO

mode, the controller combines the ML output with fixed safety rules, shown in Table 4.2.

**Table 4.2. AUTO-mode control rules**

| Condition | Control action | Type |
|---|---|---|
| Moisture ≤ 14% (target) [confirm the moisture value source, since no moisture sensor is installed] | STOP command; push notification sent to the device owner | Termination |
| Temperature > 70 °C | EMERGENCY_STOP | Safety |
| Temperature > 65 °C | Heater turned off | Safety |
| Relative humidity > 75% | Fan speed increased [confirm: with ON/OFF relays, state what this action does] | Reactive |
| Predicted drying rate / remaining time | [describe rule, e.g., adjust heater and fan when drying is slower or faster than predicted] | Predictive |

A two-minute cooldown between automatic commands prevents command flooding. Each automatic action is recorded in the Firebase predictiveControl node and broadcast through Server-Sent Events to the admin dashboard. The heater thermostat band is executed in the Arduino firmware (Section 4.1.4), and the hardware tests in Section 4.5 exercised that thermostat control. [Add test evidence for the 65 °C and 70 °C safety cut-offs and for the predictive rule, if available.]

### *4.1.3 Moisture Data Pipeline*

A software pipeline for moisture data was implemented. A moisture value in the sensor payload is accepted by POST /api/v1/sensors/data (unversioned requests from the ESP8266 are rewritten by the backend middleware), stored in MongoDB Atlas (SensorDatas collection), mirrored to Firebase Realtime Database (grain/devices/{deviceId}/sensors), and used to update the active drying session. The mobile application receives the value through Firebase onValue listeners, and the same value is passed to the ML service and the control loop.

The current prototype hardware does not include a moisture sensor (Section 4.2). [State the moisture input source used in testing: manual entry, simulated value, or estimate from time and temperature.] The pipeline therefore prepares the system for full moisture-sensor integration; it does not by itself demonstrate measured grain moisture.

[INSERT SCREENSHOT: dashboard showing live temperature, humidity, and moisture values]

*Figure 4.2. Real-time monitoring on the mobile application*

### *4.1.4 Automatic Fan and Heater Control*

Fan and heater operation is decided by the control loop from the current temperature and humidity readings and the predicted drying condition. The relay modules perform ON/OFF power switching only, using the command contract in Table 3.2 (for example, FAN:FAN1:ON and H1:1). In the implemented firmware, both fans stay energized for the whole drying session and no proportional fan speed is implemented at the relay level. The Arduino regulates the heater with a ±2 °C thermostat band around the app-set target temperature (default 45 °C). [Explain how the control loop's fan decisions map to relay commands, since the firmware keeps both fans on during a session.]

Table 4.3 shows the behavior observed in a 30-minute MANUAL-mode run with a 45 °C target.

**Table 4.3. Heater and fan behavior (30-minute run, target 45 °C)**

| Elapsed time | Chamber temp (°C) | Heater | Fan 1 | Fan 2 | Observation |
|---|---|---|---|---|---|
| 0:00 | 43.0 | ON | ON | ON | Session start, heating toward target |
| 0:04 | 46.9 | ON | ON | ON | Rising normally |
| 0:08 | 47.1 | OFF | ON | ON | Upper band reached, heater cut out |
| 0:12 | 45.8 | OFF | ON | ON | Slow natural cooling |
| 0:16 | 44.1 | OFF | ON | ON | Fans circulating air |
| 0:20 | 42.9 | ON | ON | ON | Lower band reached, heater re-engaged |
| 0:24 | 45.5 | ON | ON | ON | Rising again |
| 0:28 | 47.0 | OFF | ON | ON | Second cut-out |

The chamber stayed within 42.9–47.1 °C, which is consistent with the ±2 °C hysteresis around the 45 °C target. The log shows two heater cut-outs (0:08 and 0:28) and one re-engagement (0:20). [CONFIRM: the hardware write-up says three complete on/off cycles; make the text match the raw log.] Both fans stayed energized throughout, matching the binary ON/OFF fan control in the firmware.

### *4.1.5 Additional Implemented Features*

- Authentication with Email OTP and SMS OTP verification; unverified accounts are blocked from logging in.
- QR-based device claiming with a one-time hashed token and one device per farmer.
- Weather integration (OpenWeatherMap) with pre-session suggestions that pre-fill the drying settings.
- Analytics and PDF session reports with grain quality grading (A/B/C/F).
- grAIn Assistant powered by Google Gemini, with access to live sensors, session history, ML predictions, and dryer commands.

### *4.1.6 Agile Iterative Development*

Development followed Agile Iterative sprints (P2-A to P2-J), in which each increment was implemented, tested, and reviewed before the next sprint began. [Insert sprint table: sprint code, scope, outcome, and dates.]

## 4.2 Hardware Implementation

This section presents the physical prototype that was built and tested.

### *4.2.1 Hardware Components*

**Table 4.4. Hardware components used in the system**

| Component | Specification / model | Qty | Purpose and function |
|---|---|---|---|
| Arduino UNO | ATmega328P, 16 MHz | 1 | Main controller. Reads the DHT22 every 2 s, executes commands from the ESP8266 (START, STOP, fan, stepper), drives the relay outputs, and replies with ACK:OK or ACK:ERROR. |
| ESP8266 (NodeMCU) | ESP-12E | 1 | WiFi bridge. Polls the backend for pending commands every 10 s, forwards them to the Arduino at 9600 baud, posts readings to the backend, and reports acknowledgements. |
| DHT22 sensor | AM2302 | 1 | Measures chamber temperature (°C) and relative humidity (%). Read every 2 s. |
| Relay module | 4-channel, LOW-level trigger | 1 | Switches the heater, Fan 1, Fan 2, and the stirring mechanism on or off from Arduino signals. |
| Heater | 100 W heating element, 220 V AC (switched via relay) | 1 | Provides drying heat to the rice sample. |
| Fan 1 | 12 V DC fan | 1 | Primary airflow through the drying chamber. |

| Component | Specification / model | Qty | Purpose and function |
|---|---|---|---|
| Fan 2 | 12 V DC fan | 1 | Secondary airflow and circulation. |
| Stirring mechanism | Stepper motor [confirm model and driver] | 1 | Stirs the grain during the drying session (STEP:CW / STEP:CCW). |
| Drying chamber / tray | 40 × 30 × 15 cm tray, 2.0 kg rice capacity | 1 | Holds the rice sample during the drying run. |
| Power supplies | 12 V 5 A (loads and relay coils) and 5 V USB (control boards) | 1 set | Separate supplies for the control electronics and for the relay and load side. |
| Wiring, 10 kΩ pull-up resistor | — | as needed | DHT22 data pull-up and inter-board wiring. |

[INSERT PHOTO: assembled hardware prototype (chamber, Arduino, ESP8266, relay module, fans, heater)]

*Figure 4.3. grAIn hardware prototype*

### *4.2.2 Assembly, Wiring, and Configuration*

The prototype was assembled around an Arduino UNO, which reads the DHT22 sensor placed inside the drying chamber and drives the relay module directly. The ESP8266 acts only as a WiFi bridge. It is connected to the Arduino by a crossed serial link (ESP8266 TX/GPIO1 to Arduino RX, ESP8266 RX/GPIO3 to Arduino TX, common ground) at 9600 baud, and its GPIO2 drives a buzzer for status tones. Both boards were programmed with the Arduino IDE; the ESP8266 was configured with the WiFi credentials and the device ID GR-001. The relay module switches the heater and fans through its COM/NO contacts, which keeps the low-voltage control side separate from the load side. The control electronics run from a dedicated 5 V supply and the relay coils and loads from a separate 12 V supply, with a common ground. [Insert the Arduino and ESP8266 pin tables from the hardware engineer.]

**Table 4.5. Configuration values used**

| Setting | Value |
|---|---|
| Serial baud rate (both boards) | 9600 |
| DHT22 read interval | 2 s |
| Sensor transmit interval (Arduino to ESP8266) | 3 s initially; 1 s in the final build (Section 4.5.5) |
| ESP8266 command poll interval | 10 s |
| Serial PING heartbeat | 15 s |
| Arduino link timeout | 25 s |

| Setting | Value |
|---|---|
| Command retry | 3 attempts × 700 ms |
| Default target temperature | 45 °C |
| Manual thermostat hysteresis | ±2 °C |
| Backend stale-command timeout | 3 min, then auto-failed |

### *4.2.3 How the Hardware Components Work Together*

The system operates as a two-board pipeline. The Arduino UNO reads the DHT22 every two seconds and sends the reading over serial to the ESP8266, which posts it to the backend, where it is stored and relayed to the mobile application as live data.

When the user presses Start in the app, the backend queues a command such as START:MANUAL:45:80. The ESP8266 picks it up on its next 10-second poll and forwards it to the Arduino. The Arduino executes the command by energizing the heater and fans through the relay module and replies ACK:OK, which the ESP8266 reports to the backend, marking the command executed. The same flow handles STOP, FAN:FAN1:ON, and STEP:CW. In manual mode the Arduino regulates the heater with the ±2 °C thermostat band around the app-set target while both fans remain energized for the session.

[INSERT DIAGRAM: DHT22 → Arduino UNO → (serial 9600) → ESP8266 → (WiFi/HTTP) → Backend → Mobile app; commands return along the same path to the relay module]

*Figure 4.4. Hardware data and command flow*

## 4.3 Model Development Results

### *4.3.1 Training Dataset*

A standardized real drying dataset for this dryer configuration was not available. The training data were therefore generated from Page's thin-layer equation, MR = exp(−k · t^n), with equilibrium moisture from the Henderson–Thompson equation, an approach supported by Mohanty et al. (2021). Table 4.6 summarizes the dataset.

**Table 4.6. Synthetic dataset summary**

| Parameter | Value |
|---|---|
| Simulated drying sessions | 50 |
| Raw sensor readings | 25,160 |
| Valid training samples | 22,753 |
| Train / test split (80/20) | 18,202 / 4,551 |
| Initial moisture | 18–28% wet basis |
| Temperature | 32–75 °C |
| Relative humidity | 22–91% |
| Fan speed | 21–100% |
| Solar voltage | 3–24 V |
| Maximum session duration | 452 minutes |

### *4.3.2 Random Forest Performance*

**Table 4.7. Random Forest moisture prediction (30 minutes ahead)**

| Metric | Train | Test |
|---|---|---|
| $R^2$ | 0.9913 | 0.9457 |
| RMSE (%) | 0.25 | 0.83 |
| MAE (%) | — | 0.65 |
| MAPE (%) | — | 3.57 |
| 5-fold CV $R^2$ | 0.9172 ± 0.0175 | |

The test $R^2$ of 0.9457 exceeds the 0.90 acceptance criterion, and the cross-validation standard deviation of 0.0175 is below the 0.03 limit, both set in Chapter 3. The difference between the training $R^2$ (0.9913) and the test $R^2$ shows some overfitting, but the cross-validation mean of 0.9172 indicates stable generalization.

**Table 4.8. Feature importance of the Random Forest model**

| Rank | Feature | Importance |
|---|---|---|
| 1 | moisture (current reading) | 0.432 |
| 2 | moisture_diff_from_target | 0.409 |
| 3 | humidity | 0.057 |
| 4 | time_elapsed | 0.033 |
| 5 | fan_speed | 0.020 |
| 6 | energy_consumed | 0.019 |
| 7 | temperature | 0.014 |

| Rank | Feature | Importance |
|---|---|---|
| 8 | solar_voltage | 0.011 |
| 9 | drying_rate | 0.005 |

Current moisture and moisture difference from the target together account for 84.1% of the importance, which agrees with Smith et al. (2022), who reported 78% for the same two features. This is physically reasonable: the next moisture value depends mainly on where the grain is now and how far it is from the 14% target.

### *4.3.3 Multi-Model Comparison*

**Table 4.9. Comparison of prediction approaches (test set)**

| Model | Type | $R^2$ | RMSE (%) | MAE (%) | MAPE (%) |
|---|---|---|---|---|---|
| Multiple Linear Regression (baseline) | Data-driven | 0.7821 | 2.14 | 1.72 | — |
| Random Forest | Data-driven | 0.9457 | 0.83 | 0.65 | 3.57 |
| Gradient Boosting [confirm model] | Data-driven | [ ] | [ ] | [ ] | [ ] |
| Modified Chung–Pfost | Physics-based (EMC) | [ ] | [ ] | [ ] | [ ] |

[Discuss which model performs best and why, and how combining the data-driven and physics-based estimates affects the final prediction.] Because all models are trained or evaluated on synthetic data, the high scores show how closely the models reproduce the generating equations. They are not proof of field accuracy. Section 4.4 therefore checks predictions against completed drying sessions. The model also uses moisture as its strongest input, and the prototype has no moisture sensor, so the source of that input must be stated when reporting validation results.

## 4.4 Model Validation Using Completed Drying Sessions

For each completed drying session, the predicted drying time at [state when the prediction was taken, e.g., start of session or 30 minutes in] was compared with the actual drying time. Accuracy was computed as: Accuracy (%) = (1 − |T_pred − T_actual| / T_actual) × 100. [State the moisture input source used for these sessions.]

**Table 4.10. Predicted versus actual drying time**

| Session | Initial moisture (%) | Predicted (h:min) | Actual (h:min) | Abs. error (min) | Accuracy (%) |
|---|---|---|---|---|---|
| [S1] | [ ] | [ ] | [ ] | [ ] | [ ] |
| [S2] | [ ] | [ ] | [ ] | [ ] | [ ] |
| [S3] | [ ] | [ ] | [ ] | [ ] | [ ] |
| Mean | | | | [ ] | [ ] |

[Discuss the results: mean accuracy, the largest error and its likely cause, and whether predictions improved as the session progressed.]

## 4.5 System Performance and Integration Testing

### *4.5.1 Test Procedures*

Seven hardware and integration tests were run against the acceptance criteria in Table 3.4. Table 4.11 summarizes the procedures.

**Table 4.11. Hardware and integration test procedures**

| Test | Procedure | Criterion |
|---|---|---|
| 1. Sensor accuracy | DHT22 compared with a reference thermometer and hygrometer; 2 conditions (ambient, heated) × 3 trials | ±1 °C; ±3% RH |
| 2. Relay switching | Send H1:1, H1:0, FAN:ALL:ON/OFF, FAN:FAN1:ON, FAN:FAN2:ON; measure command-to-relay-click time, 3 trials each | ≤ 500 ms |
| 3. Command response | Press Start, Stop, fan, and heater in the app and time the actuator response, 3 trials each | ≤ 30 s round trip |
| 4. Sensor-to-mobile latency | Note the sensor reading time and the time the same value appears on the phone, 3 trials | ≤ 2 s |
| 5. Automatic thermostat | MANUAL mode, 45 °C target, 30-minute run; log every heater transition with chamber temperature | ±2 °C band |
| 6. Full drying run | 2.0 kg rice, 60-minute session; log time, temperature, RH, heater/fan/stirrer states, app agreement, and anomalies every 10 minutes | Actuators operate as commanded |
| 7. Communication reliability | 30-minute window; count polls, dispatches, executed/failed/superseded commands, sensor posts, and resets | — |

### *4.5.2 Temperature and Humidity Accuracy*

**Table 4.12. DHT22 accuracy versus reference instrument (2 conditions × 3 trials)**

| Trial | Condition | Ref temp (°C) | DHT22 (°C) | Error (°C) | Ref RH (%) | DHT22 RH (%) | Error (%) |
|---|---|---|---|---|---|---|---|
| 1 | Ambient | 27.4 | 27.8 | +0.4 | 61 | 63 | +2 |

| Trial | Condition | Ref temp (°C) | DHT22 (°C) | Error (°C) | Ref RH (%) | DHT22 RH (%) | Error (%) |
|---|---|---|---|---|---|---|---|
| 2 | Ambient | 27.6 | 27.3 | −0.3 | 62 | 60 | −2 |
| 3 | Ambient | 27.5 | 27.9 | +0.4 | 61 | 63 | +2 |
| 4 | Heated | 44.8 | 44.3 | −0.5 | 48 | 46 | −2 |
| 5 | Heated | 48.2 | 48.9 | +0.7 | 45 | 47 | +2 |
| 6 | Heated | 51.0 | 50.5 | −0.5 | 43 | 41 | −2 |
| Mean absolute error | | | | 0.47 | | | 2.0 |

The mean absolute temperature error was 0.47 °C (criterion ±1 °C) and the mean absolute humidity error was 2.0% (criterion ±3%). All six trials were within both criteria. The largest deviation (+0.7 °C) occurred in the heated condition, which may be due to localized warm air near the sensor. Result: Pass.

### *4.5.3 Relay Operation*

**Table 4.13. Relay switching time, command receipt at the Arduino to relay actuation (criterion ≤ 500 ms)**

| Command | Relay channel | Trial 1 (ms) | Trial 2 (ms) | Trial 3 (ms) | Mean (ms) | Result |
|---|---|---|---|---|---|---|
| H1:1 heater ON | CH1 | 96 | 104 | 99 | 100 | Pass |
| H1:0 heater OFF | CH1 | 98 | 101 | 97 | 99 | Pass |
| FAN:ALL:ON | CH2+CH3 | 112 | 108 | 115 | 112 | Pass |
| FAN:ALL:OFF | CH2+CH3 | 110 | 106 | 111 | 109 | Pass |
| FAN:FAN1:ON | CH2 | 105 | 109 | 103 | 106 | Pass |
| FAN:FAN2:ON | CH3 | 107 | 112 | 108 | 109 | Pass |

All relay activations occurred within 96–115 ms (overall mean 106 ms), well below the 500 ms criterion. The audible click and the load state always matched the command sent, and no relay chatter or missed switching was observed across 18 trials. Result: Pass.

### *4.5.4 Command Response from the Application*

**Table 4.14. Command response, app press to actuator (mean of 3 trials each; criterion ≤ 30 s)**

| Command | Observed response time (s) | ACK returned to backend | Result |
|---|---|---|---|
| Start session | 4.6 | Yes — executed | Pass |
| Stop session | 4.2 | Yes — executed | Pass |
| Fan ON | 3.4 | Yes — executed | Pass |
| Fan OFF | 2.7 | Yes — executed | Pass |
| Heater ON | 4.8 | Yes — executed | Pass |
| Mean | 3.9 [CONFIRM: the write-up says 3.8; the table values average 3.94] | — | Pass |

Response times ranged from 2.7 s to 4.8 s, well inside the 30 s criterion. The delay is dominated by the ESP8266's 10-second command poll interval: a command queued just before a poll is picked up quickly, while one queued just after a poll waits for the next window, plus roughly 0.5–1.5 s of backend and HTTP latency. This is a property of polling-based WiFi control and not a fault. It could be reduced with a shorter poll interval or a push-based channel.

**Table 4.15. Hardware response to commands from the application**

| App action | Hardware response observed | Working? |
|---|---|---|
| Start session | Heater and both fans energized via relays | Yes |
| Complete session | All loads de-energized; session saved in History | Yes |
| Abort session | Loads de-energized immediately | Yes |
| Toggle Fan 1 | Fan 1 relay toggled | Yes |
| Toggle Fan 2 | Fan 2 relay toggled | Yes |
| Toggle heater | Heater relay toggled | Yes |
| Adjust target temperature | Arduino thermostat re-targeted at next reading | Yes |

### *4.5.5 Sensor-to-Mobile Latency*

**Table 4.16. Sensor-to-mobile latency (criterion ≤ 2 s)**

| Trial | Reading to visible on phone (s) | Result |
|---|---|---|
| 1 | 1.8 | Pass |
| 2 | 2.6 | Fail |
| 3 | 3.2 | Fail |
| Mean (initial) | 2.5 | Fail |

| Trial | Reading to visible on phone (s) | Result |
|---|---|---|
| Retest after setting the transmit interval to 1 s (1.1 / 0.9 / 1.2) | 1.1 | Pass |

The initial mean latency of 2.5 s exceeded the 2 s criterion because the firmware transmitted sensor data every 3 seconds, so a reading could wait up to 3 s before it was posted. After the transmit interval was reduced to 1 second and the test repeated, the mean dropped to 1.1 s and met the criterion. The 1-second interval is kept in the final firmware build.

### *4.5.6 Communication Reliability*

**Table 4.17. 30-minute communication reliability**

| Metric | Count |
|---|---|
| ESP8266 backend polls | 180 |
| Commands dispatched to Arduino | 12 |
| Commands acknowledged (executed) | 10 |
| Commands failed (no ACK within 3 min) | 1 |
| Commands superseded (replaced by a newer command) | 1 |
| Sensor posts attempted / HTTP 200 | 600 / 598 |
| Board resets or WiFi dropouts | 0 [CONFIRM: see text] |

Ten of twelve commands executed normally. The superseded command was a fan toggle replaced by a later command by design, since the backend supersedes older queued commands so that one button press results in one execution. The failed command is attributed to a WiFi loss longer than the backend's 3-minute acknowledgement timeout. [CONFIRM: the table reports 0 WiFi dropouts and only 2 failed posts, which does not fit a WiFi loss of more than 3 minutes. Correct the cause or the count from the raw log.] Sensor posting succeeded in 99.7% of attempts; the two failures coincided with a Render.com cold start and were retried by the next transmit cycle. No board resets occurred in the window.

### *4.5.7 Full Drying Run*

**Table 4.18. Full drying run log, 2.0 kg rice, 60 minutes**

| Elapsed | Temp (°C) | RH (%) | Heater | Fan 1 | Fan 2 | Stirrer | App matches ? | Notes |
|---|---|---|---|---|---|---|---|---|
| 0:00 | 28.6 | 62 | ON | ON | ON | ON | Yes | Session started |
| 0:10 | 34.2 | 58 | ON | ON | ON | ON | Yes | — |
| 0:20 | 39.5 | 55 | ON | ON | ON | ON | Yes | — |
| 0:30 | 43.1 | 51 | ON | ON | ON | ON | Yes | — |
| 0:40 | 45.6 | 48 | OFF | ON | ON | ON | Yes | Thermostat cut-out [CONFIRM: cut-out at 45.6 °C does not match the 47 °C upper band] |
| 0:50 | 46.8 | 45 | ON | ON | ON | ON | Yes | — |
| 1:00 (end) | 47.2 | 43 | OFF | ON | ON | ON | Yes | Session completed normally |

During the 60-minute run, temperature rose from 28.6 to 47.2 °C and relative humidity fell from 62 to 43%. All actuators operated as commanded, and the mobile app showed values consistent with the chamber at every logged interval. No resets, dropped readings, or failed commands were observed. Because the prototype has no moisture sensor, the run shows the temperature and humidity behavior of the chamber and not a measured drop in grain moisture. [State whether the grain was weighed or oven-tested before and after the run.]

### *4.5.8 Summary Against the Acceptance Criteria*

**Table 4.19. Hardware performance versus acceptance criteria**

| Test | Criterion | Measured | Result |
|---|---|---|---|
| Temperature accuracy vs reference | ±1 °C | 0.47 °C mean abs. error | Pass |
| Humidity accuracy vs reference | ±3% | 2.0% mean abs. error | Pass |
| Relay switching time | ≤ 500 ms | 106 ms mean | Pass |
| ESP8266 to backend POST latency | ≤ 5 s (warm) | 0.64 s mean (max 1.9 s); one 42 s cold start | Pass |
| Sensor update shown on mobile | ≤ 2 s | 2.5 s initial, 1.1 s after firmware change | Fail, then Pass on retest |
| Command acknowledgement round trip | ≤ 30 s | 3.9 s mean (2.7–4.8 s) [CONFIRM mean] | Pass |

Five of the six criteria were met on the first run. The sensor-update criterion failed at first because of the firmware's 3-second transmit interval and not because of a hardware defect; reducing the interval to 1 second gave 1.1 s on retest. The

command round trip averages about 3.9 s because of the 10-second poll architecture, which is well inside the 30 s criterion but adds noticeable delay in interactive use. The single 42-second backend delay was a Render.com free-tier cold start; the keep-alive job prevents most such events, and no data was lost because sensor posts retry on the next cycle. After the firmware adjustment, all hardware criteria were met. Moisture, load-cell weight, and solar-voltage sensing were not tested because those sensors are not part of the hardware scope.

### 4.6 ISO/IEC 25010 Evaluation

The system was evaluated by experts and end-users using a questionnaire based on eight ISO/IEC 25010 quality characteristics: Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, and Portability. Each characteristic has five statements rated on a five-point scale. The expert and end-user versions measure the same characteristics, with wording suited to each group.

#### *4.6.1 Respondents*

**Table 4.20. Profile of respondents**

| Group | Description | n |
|---|---|---|
| Experts | [e.g., IT faculty, software and embedded-systems professionals] | [ ] |
| End-users | [e.g., rice farmers or target users] | [ ] |
| Total | | [ ] |

#### *4.6.2 Rating Scale and Interpretation*

**Table 4.21. Five-point scale and verbal interpretation**

| Scale | Mean range | Verbal interpretation |
|---|---|---|
| 5 | 4.21 – 5.00 | Strongly Agree |
| 4 | 3.41 – 4.20 | Agree |
| 3 | 2.61 – 3.40 | Neutral |
| 2 | 1.81 – 2.60 | Disagree |
| 1 | 1.00 – 1.80 | Strongly Disagree |

### *4.6.3 Results by Quality Characteristic*

**Table 4.22. Mean ratings per ISO/IEC 25010 characteristic**

| Characteristic | Expert mean | Interpretation | End-user mean | Interpretation |
|---|---|---|---|---|
| Functional Suitability | [ ] | [ ] | [ ] | [ ] |
| Performance Efficiency | [ ] | [ ] | [ ] | [ ] |
| Compatibility | [ ] | [ ] | [ ] | [ ] |
| Usability | [ ] | [ ] | [ ] | [ ] |
| Reliability | [ ] | [ ] | [ ] | [ ] |
| Security | [ ] | [ ] | [ ] | [ ] |
| Maintainability | [ ] | [ ] | [ ] | [ ] |
| Portability | [ ] | [ ] | [ ] | [ ] |
| Overall mean | [ ] | [ ] | [ ] | [ ] |

Item-level means are presented in Appendix [ ]. The sections below relate each rating to features of the implemented system.

***Functional Suitability***

Experts rated this characteristic [x.xx] and end-users [x.xx]. The system provides temperature and humidity monitoring, relay-based heater, fan, and stirrer control, a 30-minute moisture forecast, and time-to-target estimation.

***Performance Efficiency***

Ratings were [x.xx] (experts) and [x.xx] (end-users). Sensor values reached the app in 1.1 s on average after the firmware change, and commands took 2.7–4.8 s end to end (Tables 4.14 and 4.16). A keep-alive job limits Render.com cold-start delays.

***Compatibility***

Ratings were [x.xx] and [x.xx]. The ESP8266, Arduino, Next.js backend, Flask ML service, Firebase, and the mobile app exchange data through defined REST and command contracts, and 598 of 600 sensor posts succeeded in the reliability test.

***Usability***

Ratings were [x.xx] and [x.xx]. The app follows a dashboard-first design, with alerts and push notifications for key drying events.

***Reliability***

Ratings were [x.xx] and [x.xx]. A rule-based fallback runs when the ML service is unreachable, an offline command queue holds commands during connection loss, and the full 60-minute run completed with no resets or dropped readings.

***Security***

Ratings were [x.xx] and [x.xx]. The system uses JWT access and refresh tokens, role-based access control, OTP-verified accounts, and token storage in Expo SecureStore.

***Maintainability***

Ratings were [x.xx] and [x.xx]. The system is split into separate backend, ML service, and mobile repositories with versioned API endpoints.

***Portability***

Ratings were [x.xx] and [x.xx]. The Expo React Native app targets Android API level 33 and above and adapts its layout to different screen sizes.

### *4.6.4 Limitations of the Evaluation*

The evaluation involved [n] respondents ([n] experts and [n] end-users), selected through [sampling method, e.g., purposive sampling]. The results therefore describe the prototype's evaluation context and cannot be generalized to all farmers or regions. [Add adviser-approved justification for the sample size.]

## 4.7 Discussion of Results Against the Objectives

*Align the wording of the Chapter 1 objectives and the Chapter 3 hardware list with the built prototype (ESP8266 instead of ESP32; no moisture, load-cell, or solar sensing hardware) before the panel reads this table.*

**Table 4.23. Status of the specific objectives**

| Specific objective | Result | Status |
|---|---|---|
| Data acquisition of drying parameters (temperature and humidity sensed in hardware; moisture, weight, and solar voltage as software or model inputs) [align with revised Chapter 1] | DHT22 mean errors 0.47 °C and 2.0% RH (Table 4.12); sensor data reached the app in 1.1 s (Table 4.16) | Achieved for temperature and humidity [state status for the other parameters] |
| Random Forest model with test $R^2 \geq 0.90$ | Test $R^2 = 0.9457$; CV $R^2 = 0.9172 \pm 0.0175$ | Achieved |

| Specific objective | Result | Status |
|---|---|---|
| AI-driven predictive control of fan, heater, and stirring | Thermostat and relay control verified (Tables 4.3, 4.13, 4.18); predictive logic in Section 4.1.2 [add validation accuracy from Table 4.10] | [ ] |
| Solar PV subsystem with solar voltage as a model feature | Solar voltage is a model feature (importance 0.011); no solar sensing hardware was built [state how the multi-powered design is addressed] | [ ] |
| React Native Android application | Section 4.1 and Section 4.6 usability ratings | [ ] |
| Next.js web admin dashboard with SSE updates | [summarize dashboard results] | [ ] |
| Evaluation of accuracy, control responsiveness, and usability | Sections 4.4 to 4.6 | [ ] |

[Write 2 to 3 paragraphs: what worked as expected, what did not and why, and how the results compare with Kumar et al. (2022) and Smith et al. (2022).]

# CHAPTER 5

## SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

*DRAFT NOTE (delete these niggas before submission): text highlighted in yellow depends on results still being collected. Update it after Chapter 4 is final.*

This chapter summarizes the findings of the study, presents the conclusions drawn from them, states the limitations, and gives recommendations for improving the grAIn system and for future research.

### 5.1 Summary of Findings

The study developed and evaluated grAIn, an AI-assisted multi-powered grain drying system with Android-based monitoring and predictive control. The main findings are as follows.

#### *5.1.1 Hardware Implementation and Testing*

The prototype consists of an Arduino UNO reading a DHT22 temperature and humidity sensor every two seconds and driving a 4-channel relay module that switches the heater, Fan 1, Fan 2, and the stirring mechanism inside a 40 × 30 × 15 cm drying chamber holding 2.0 kg of rice. An ESP8266 acts as the WiFi bridge: it polls the backend every 10 seconds for queued commands, forwards them to the Arduino at 9600 baud, and posts sensor readings for display in the mobile application. Every command is confirmed through an ACK:OK handshake that the backend records as executed.

Hardware testing covered six acceptance criteria: sensor accuracy, relay switching time, command response, sensor-to-mobile latency, automatic thermostat control, and full-system operation (Table 4.19). The DHT22 had mean absolute errors of 0.47 °C and 2.0% RH. Relays switched in 106 ms on average. Commands reached the hardware in 2.7–4.8 s against a 30 s criterion. The full 60-minute run finished with no resets, dropped readings, or failed commands. Five of six criteria passed on the first run, and all six passed after the sensor transmit interval was reduced to 1 second.

### *5.1.2 Prediction and Control*

- Prediction model. A Random Forest model trained on a 22,753-sample synthetic dataset reached a test $R^2$ of 0.9457, RMSE of 0.83%, MAE of 0.65%, and MAPE of 3.57%, with a 5-fold cross-validation $R^2$ of 0.9172 ± 0.0175. These values meet the Chapter 3 criteria ($R^2 \geq 0.90$; CV standard deviation < 0.03). Current moisture and moisture difference from target were the strongest predictors (84.1% combined importance).
- Multi-model prediction. Gradient Boosting and the Modified Chung–Pfost EMC model were added to the Random Forest. [State which model performed best and the key figures from Table 4.9.]
- Predictive control. Predicted drying rate and remaining drying time were integrated into the backend control loop, which also keeps safety limits (emergency stop above 70 °C, heater off above 65 °C). On the hardware side, the Arduino held the chamber within 42.9–47.1 °C of a 45 °C target using a ±2 °C thermostat band, and fans were switched ON/OFF through relays. [Add predictive-control validation evidence.]
- Moisture data pipeline. The software pipeline from the backend through MongoDB and Firebase to the mobile application accepts moisture values and passes them to the prediction and control functions. The prototype has no physical moisture sensor, so moisture input came from [state source].
- Model validation. Predicted drying time compared with the actual drying time of completed sessions gave a mean accuracy of [xx.x]% across [n] sessions.

### *5.1.3 User Evaluation*

- ISO/IEC 25010 evaluation. Experts (n = [ ]) gave an overall mean of [x.xx] ([interpretation]) and end-users (n = [ ]) gave [x.xx] ([interpretation]). The highest rated characteristic was [ ] and the lowest was [ ].

## 5.2 Conclusions

Based on the findings, the study concludes the following.

- Sensor suitability. The DHT22's mean errors (0.47 °C and 2.0% RH) were within both criteria, so it is suitable for monitoring drying temperature and humidity in this prototype.

- Relay and actuator control. Relays switched in 96–115 ms across 18 trials, far below the 500 ms limit, with no chatter or missed switching. Relay-based ON/OFF control of the heater, fans, and stirrer was reliable during testing.
- Command responsiveness. End-to-end response averaged [3.9] s (maximum 4.8 s) against the 30 s criterion, and the delay comes from the 10-second poll interval. The polling design supports effective control of the dryer, but a response under 2 seconds would need a push-based or faster-poll design.
- Automatic regulation. The thermostat kept the chamber within 42.9–47.1 °C of the 45 °C target over a 30-minute run. Automatic temperature regulation worked as intended at laboratory scale.
- System integration. In the 30-minute observation, all 12 commands were accounted for (10 executed, 1 superseded by design, 1 failed) and 99.7% of 600 sensor posts succeeded. The hardware, backend, and application chain operated reliably, and the recorded failures were linked to network conditions and not to hardware defects. [Confirm the cause of the failed command against the raw log.]
- Prediction. The Random Forest model, trained on physics-based synthetic data, met the target accuracy for 30-minute moisture prediction on held-out synthetic data. Its accuracy on real drying sessions is shown by the validation result of [xx.x]% mean accuracy.
- Multi-model approach. Combining data-driven models with a physics-based EMC model gives the system more than one basis for estimating drying time and moisture behavior. [Confirm after Table 4.9.]
- User evaluation. The system was rated [interpretation] by experts and [interpretation] by end-users across the eight ISO/IEC 25010 characteristics, which [supports / partly supports] its readiness as a prototype for rice drying.
- Prototype status. The system is a proof of concept. The hardware validates the concept at laboratory scale and is not yet hardened for field deployment.

### 5.3 Limitations

- ON/OFF-only fan control. Fans are switched by binary relays, so there is no proportional airflow.

- Poll-based command latency. The 10-second poll window adds delay (about 3–5 s observed in testing).
- Single sensing point. One DHT22 measures the chamber, and the temperature inside the grain mass may differ.
- No direct grain-moisture sensor. Drying progress is judged from time and temperature behavior and from the moisture input described in Section 4.1.3, not from measured grain moisture.
- WiFi dependence. Outside WiFi range the ESP8266 cannot receive commands or forward readings.
- Power-supply sensitivity. The design depends on clean control-side power (see Table 5.1, problem 1).
- Synthetic training data. The models were trained on synthetic data and validated on [n] completed sessions.
- Laboratory scale. Tests used a 2.0 kg batch under controlled indoor conditions and were not validated for field or harvest use.

### 5.4 Hardware Problems Encountered During Testing

*Keep only the problems that actually occurred in your tests, with the real outcomes. Delete any row that did not happen.*

**Table 5.1. Hardware problems log**

| # | Problem observed | Cause (if identified) | Action taken | Status |
|---|---|---|---|---|
| 1 | DHT22 readings stopped or froze when the fans and heater switched on | Voltage dip or electrical noise on the shared 5 V rail at relay inrush | Moved control electronics to a dedicated 5 V supply (common ground), added 1000 µF bulk and 100 µF decoupling capacitors, staggered relay activation | Resolved; 60-minute run with no dropouts |
| 2 | Sensor data stopped reaching the app | Loose ESP8266–Arduino serial jumper (TX/RX) | Reseated and secured the wiring; verified common ground | Resolved; no recurrence |
| 3 | Commands showing failed or superseded in command history | Superseded: replaced by a newer command (by design). Failed: no ACK within 3 min during a WiFi drop | No fix needed for superseded; WiFi coverage improved for failed | Understood / resolved |
| 4 | One 42-second backend delay | Render.com free-tier cold start | Keep-alive cron job; sensor posts retry automatically | Resolved |

St. Dominic Complex, E. Aguinaldo Highway, Bacoor City, Cavite, Philippines 4102
0917 625 8222 • 0922 850 9200 • (046) 417 7322 • (046) 417 8161
www.stdominiccollege.edu.ph stdominiccollege @sdcapikemen


## 5.5 Recommendations

### *5.5.1 Model and Data*

- Retrain the models on real drying sessions from the prototype (20 to 30 complete cycles or more) and compare against the synthetic-trained baseline.
- Add online or periodic retraining so the models improve as new sessions are completed.
- Build separate models for other grain types (for example corn and wheat) and for different batch sizes.
- Extend the model to predict optimal temperature and fan settings, not only future moisture.

### *5.5.2 Hardware and Sensing*

- Add a dedicated grain-moisture sensor (target accuracy of ±0.5% wet basis), calibrated against the oven-dry method, so the drying endpoint is measured and not estimated.
- Keep the dedicated, regulated control-side power arrangement (separate supply for the boards, bulk and decoupling capacitors, common ground, staggered relay activation).
- Secure and protect the serial link: strain-relief the ESP8266–Arduino wiring, add a voltage divider on the ESP8266 RX line (3.3 V input), and add a visible link-lost indicator.
- Replace the ON/OFF fan relays with proportional control (PWM or a speed setpoint) for stage-based airflow.
- Shorten the poll interval or use a push channel if near-real-time control is needed.
- Add an enclosure, strain relief, and proper fusing for the heater and mains side before any field trial.
- Add a load cell for weight-loss cross-checking of moisture, and solar PV input with voltage and current sensing for solar-aware control.
- Consider a local LCD or OLED status display, battery back-up, and a LoRa link for farms beyond WiFi range.

### *5.5.3 Software and Deployment*

- Move the control loop to a dedicated worker and use a paid hosting plan to avoid cold starts and free-tier limits.
- Add ONNX export for on-device prediction, so basic predictions still work when the internet is down.
- Provide an iOS build if farmers or technicians request it.

### *5.5.4 Future Research*

- Run field pilots with larger batches across seasons and regions to move beyond laboratory validation.
- Run a comparative study of AI auto-stop against manual monitoring over a larger number of drying sessions.
- Repeat the ISO/IEC 25010 evaluation with a larger and more varied group of experts and farmers.
- Study the energy savings from solar-aware control using measured solar and battery data.